07

ETL vs ELT

Xây pipeline đầu tiên bằng Python

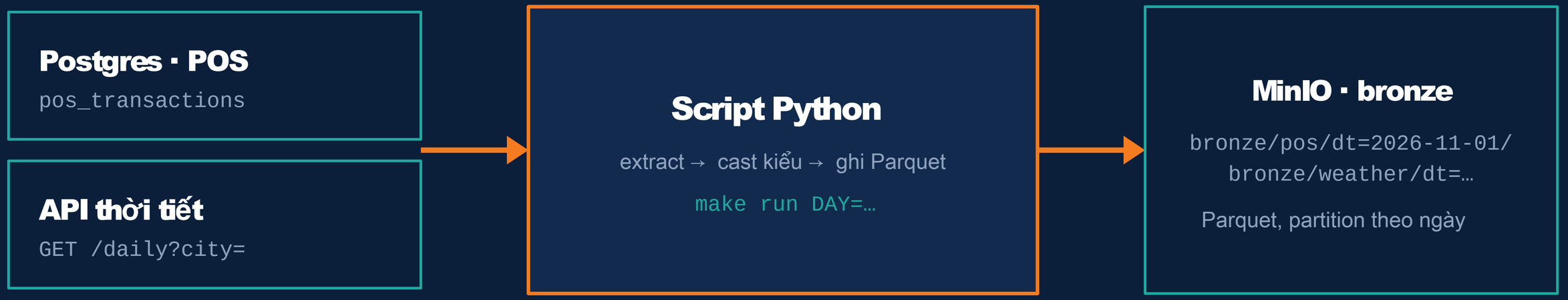
Hôm nay mình nói gì

1. ETL vs ELT
2. Extract từ DB, API, file
3. Incremental load & watermark
4. Idempotency · Retry, backoff
5. Cấu trúc project

ETL vs ELT

	ETL	ELT
Biến đổi ở đâu	Trước khi load, máy trung gian	Sau khi load, trong warehouse
Phù hợp	Dữ liệu nhạy cảm, ít	Dữ liệu lớn, warehouse mạnh
Công cụ	Python, Spark	dbt, SQL
Xu hướng	Giảm dần	Mặc định với cloud warehouse

Pipeline đầu tiên của dự án



Nhỏ, chạy được mỗi ngày, trước khi nghĩ đến Spark.

Extract từ Postgres, chỉ phần mới

```
import polars as pl

QUERY = """
    SELECT order_id, store_id, product_id, amount, ts
    FROM pos_transactions
    WHERE ts >= :since AND ts < :until
"""

def extract_pos(conn, since, until) -> pl.DataFrame:
    return pl.read_database(
        QUERY, conn, execute_options={"parameters": {"since": since, "until": until}}
    )
```

Không bao giờ kéo cả bảng mỗi ngày.

Full load vs incremental

- Full = kéo hết, đơn giản, đắt dần
- Incremental = kéo phần mới theo watermark
- Watermark = mốc cuối cùng đã xử lý
- Lưu watermark ngoài pipeline: bảng state hoặc file

Watermark trong cùng transaction

```
def run(conn, day: date) -> None:
    with conn.begin():
        since = get_watermark(conn, "pos")        # pipeline_state
        until = datetime.combine(day + timedelta(1), time.min)

        df = extract_pos(conn, since, until)
        write_parquet(df, f"bronze/pos/dt={day}/")

        new_wm = df["ts"].max() if df.height else since
        set_watermark(conn, "pos", new_wm)
        log.info("watermark updated %s -> %s", since, new_wm)
```

Cập nhật watermark sau khi ghi thành công.

Idempotency: chạy 2 lần như chạy 1 lần

- Ghi đè partition theo ngày thay vì append
- Khoá tự nhiên để upsert
- Không dùng `now()` làm dữ liệu
- Chết giữa chừng, chạy lại không trùng

Chạy lại an toàn



Partition theo ngày là đơn vị chạy lại.

Retry với exponential backoff

```
from tenacity import retry, stop_after_attempt, wait_exponential
import httpx

@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=2, max=30),
    reraise=True,
)
def fetch_weather(city: str, day: date) -> dict:
    r = httpx.get(WEATHER_URL, params={"city": city, "date": day}, timeout=10)
    r.raise_for_status()
    return r.json()
```

API ngoài luôn lỗi lúc nào đó. Chờ 2s → 4s → 8s → 16s → 30s.

Cấu trúc project pipeline

`src/ingestion/pos.py`	**Extract Postgres**
`src/ingestion/weather.py`	**Gọi API, có retry**
`src/common/state.py`	**Đọc / ghi watermark**
`tests/`	**pytest với dữ liệu giả**
`Makefile`	`make run DAY=2026-11-01`

Lỗi hay gặp

SAI	ĐÚNG
~~Append rồi xử lý trùng sau~~	Ghi đè partition
~~Watermark = thời điểm chạy~~	Watermark = `max(ts)` của dữ liệu đã lấy
~~Bắt mọi exception rồi tiếp tục~~	Fail rõ, để orchestrator retry
~~Hard-code mật khẩu~~	Biến môi trường

Lab #07 — Pipeline POS + thời tiết → Parquet

1. `make run DAY=2026-11-01`
2. **Kiểm tra Parquet trên MinIO:** `bronze/pos/dt=2026-11-01`
3. **Chạy lại cùng ngày, xác nhận không trùng**
4. **Tắt mạng giữa chừng, xem retry**

KẾT QUẢ MONG ĐỢI

2 lần chạy cho cùng số dòng

Log có dòng `watermark updated`

CÂU HỎI PHỎNG VẤN #07

Idempotent pipeline là gì, làm sao đảm bảo?

Chạy lại không đổi kết quả; ghi đè partition, upsert theo khoá.

VIDEO TIẾP THEO

08

Định dạng file dữ liệu: Parquet, Avro, ORC, Delta

Thứ Sáu 27/11 · 19:00

Đăng ký